

Homework 0: Algorithmic Thinking + NumPy Basics

Intro to Scientific Computing in Python

Instructions. Submit a short PDF or a well-organized notebook (or both) that includes: (1) your final code, (2) a brief explanation of your approach, and (3) the time/space complexity for each LeetCode problem. For the NumPy exercise, show intermediate outputs (or clearly label results).

Collaboration policy. You may discuss high-level ideas with classmates, but your code and write-up must be your own.

Part A: LeetCode (Medium) — Choose a clean, efficient solution

1. Group Anagrams (Medium)

Problem. Given an array of strings `strs`, group the anagrams together. You can return the answer in any order.

Example.

- Input: `["eat", "tea", "tan", "ate", "nat", "bat"]`
- Output (one valid): `[["bat"], ["nat", "tan"], ["ate", "eat", "tea"]]`

Requirements.

- Aim for $O(n \cdot k)$ time where n is the number of strings and k is the average string length.
- Use a dictionary/hash map to group items.
- In your write-up: explain what you use as a dictionary key (and why).

Link. [LeetCode: Group Anagrams](#)

2. Top K Frequent Elements (Medium)

Problem. Given an integer array `nums` and an integer `k`, return the `k` most frequent elements.

Example.

- Input: `nums = [1,1,1,2,2,3]`, `k = 2`
- Output: `[1,2]`

Requirements.

- First build a frequency dictionary.
- Then choose an efficient strategy to extract the top k (avoid sorting the entire list if possible).
- In your write-up: describe the strategy you used (e.g., bucket idea or heap idea) and give time/space complexity.

Link. [LeetCode: Top K Frequent Elements](#)

Part B: NumPy Homework Exercise (Scientific Computing Mini-Lab)

NumPy Exercise: Vectorization, Masking, and Basic Statistics

Goal. Practice the core NumPy ideas: creating arrays, vectorized computation, boolean masking, reductions, and reproducibility.

Setup. Use a reproducible random number generator:

```
import numpy as np
rng = np.random.default_rng(0)
```

Create a synthetic dataset $X \in \mathbb{R}^{1000 \times 3}$ where each row is a sample and each column is a feature:

```
X = rng.normal(loc=0.0, scale=1.0, size=(1000, 3))
```

Tasks. Complete each step using NumPy operations (no Python loops unless explicitly asked).

1. **Array inspection.** Print (or record) `X.shape`, `X.ndim`, `X.dtype`, and `X.size`.
2. **Column-wise statistics.** Compute the mean and standard deviation of each column (feature) using `axis=0`. Store them in `mu` and `sigma`.
3. **Standardization (vectorized).** Create a standardized version

$$Z = \frac{X - \mu}{\sigma}$$

using broadcasting. Verify that each column of Z has mean approximately 0 and standard deviation approximately 1.

4. **Boolean masking (filtering).** Define a mask for “outliers” where a row is considered an outlier if *any* standardized feature has absolute value greater than 2.5:

$$\exists j \in \{1, 2, 3\} \text{ such that } |Z_{ij}| > 2.5.$$

Use this mask to create:

- `Z_out` = all outlier rows
- `Z_in` = all non-outlier rows

Report how many outliers you found.

5. **Distances without loops.** Compute the Euclidean norm of each row of Z (distance from the origin in standardized space):

$$r_i = \|Z_i\|_2.$$

Store the result in a 1D array $\mathbf{r}$ of length 1000. Report the minimum, maximum, and mean of $\mathbf{r}$.

6. **(Short reflection) Why vectorization?** In 3–5 sentences: explain why tasks (3)–(5) are a good fit for NumPy and what you would lose by writing them with Python loops.
-

Submission Checklist

- For each LeetCode problem: final code + brief explanation + time/space complexity.
- For the NumPy exercise: code + outputs (or clearly labeled computed values).
- Your work should run without errors from a fresh restart.